

An HTTP cookie (also called web cookie, Internet cookie, browser cookie, or simply cookie) is a small block of data created by a web server while a user is browsing a website and placed on the user's computer or other device by the user's web browser. Cookies are placed on the device used to access a website, and more than one cookie may be placed on a user's device during a session.

Cookies serve useful and sometimes essential functions on the web. They enable web servers to store stateful information (such as items added in the shopping cart in an online store) on the user's device or to track the user's browsing activity (including clicking particular buttons, logging in, or recording which pages were visited in the past). They can also be used to save information that the user previously entered into form fields, such as names, addresses, passwords, and payment card numbers for subsequent use.

Authentication cookies are commonly used by web servers to authenticate that a user is logged in, and with which account they are logged in. Without the cookie, users would need to authenticate themselves by logging in on each page containing sensitive information that they wish to access. The security of an authentication cookie generally depends on the security of the issuing website and the user's web browser, and on whether the cookie data is encrypted. Security vulnerabilities may allow a cookie's data to be read by an attacker, used to gain access to user data, or used to gain access (with the user's credentials) to the website to which the cookie belongs (see cross-site scripting and cross-site request forgery for examples).

Tracking cookies, and especially third-party tracking cookies, are commonly used as ways to compile long-term records of individuals' browsing histories — a potential privacy concern that prompted European and U.S. lawmakers to take action in 2011. European law requires that all websites targeting European Union member states gain "informed consent" from users before storing non-essential cookies on their device.

Background

Origin of the name

The term cookie was coined by web-browser programmer Lou Montulli. It was derived from the term magic cookie, which is a packet of data a program receives and sends back unchanged, used by Unix programmers.

History

Magic cookies were already used in computing when computer programmer Lou Montulli had the idea of using them in web communications in June 1994. At the time, he was an employee of Netscape Communications, which was developing an e-commerce application for MCI. Vint Cerf and John Klensin represented MCI in technical discussions with Netscape Communications. MCI did not want its servers to have to retain partial transaction states, which led them to ask Netscape to find a way to store that state in each user's computer instead. Cookies provided a solution to the problem of reliably implementing a virtual shopping cart.

Together with John Giannandrea, Montulli wrote the initial Netscape cookie specification the same year. Version 0.9beta of Mosaic Netscape, released on 13 October 1994, supported cookies. The first use of cookies (out of the labs) was checking whether visitors to the Netscape website had already visited the site. Montulli applied for a patent for the cookie technology in 1995, which was granted in 1998. Support for cookies was integrated with Internet Explorer in version 2, released in October 1995.

The introduction of cookies was not widely known to the public at the time. In particular, cookies were accepted by default, and users were not notified of their presence. The public learned about cookies after the Financial Times published an article about them on 12 February 1996. In the same year, cookies received a lot of media attention, especially because of potential privacy implications. Cookies were discussed in two U.S. Federal Trade Commission hearings in 1996 and 1997.

The development of the formal cookie specifications was already ongoing. In particular, the first discussions about a formal specification started in April 1995 on the www-talk mailing list. A special working group within the Internet Engineering Task Force (IETF) was formed. Two alternative proposals for introducing state in HTTP transactions had been proposed by Brian Behlendorf and David Kristol respectively. But the group, headed by Kristol himself and Lou Montulli, soon decided to use the Netscape specification as a starting point. In February 1996, the working group identified third-party cookies as a considerable privacy threat. The specification produced by the group was eventually published as RFC 2109 in February 1997. It specifies that third-party cookies were either not allowed at all, or at least not enabled by default. At this time, advertising companies were already using third-party cookies. The recommendation about third-party cookies of RFC 2109 was not followed by Netscape and Internet Explorer. RFC 2109 was superseded by RFC 2965 in October 2000.

RFC 2965 added a Set-Cookie2 header field, which informally came to be called "RFC 2965-style cookies" as opposed to the original Set-Cookie header field which was called "Netscape-style cookies". Set-Cookie2 was seldom used, however, and was deprecated in RFC 6265 in April 2011 which was written as a definitive specification for cookies as used in the real world. No modern browser recognizes the Set-Cookie2 header field.

Terminology

Session cookie

A session cookie (also known as an in-memory cookie, transient cookie or non-persistent cookie) exists only in temporary memory while the user navigates a website.

Session cookies expire or are deleted when the user closes the web browser. Session cookies are identified by the browser by the absence of an expiration date assigned to them.

Persistent cookie

A persistent cookie expires at a specific date or after a specific length of time. For the persistent cookie's lifespan set by its creator, its information will be transmitted to the server every time the user visits the website that it belongs to, or every time the user views a resource belonging to that website from another website (such as an advertisement).

For this reason, persistent cookies are sometimes referred to as tracking cookies because they can be used by advertisers to record information about a user's web browsing habits over an extended period of time. Persistent cookies are also used for reasons such as keeping users logged into their accounts on websites, to avoid re-entering login credentials at every visit. (See § Uses, below.)

Secure cookie

A secure cookie can only be transmitted over an encrypted connection (i.e. HTTPS). They cannot be transmitted over unencrypted connections (i.e. HTTP). This makes the cookie less likely to be exposed to cookie theft via eavesdropping. A cookie is made secure by adding the Secure flag to the cookie.

Http-only cookie

An http-only cookie cannot be accessed by client-side APIs, such as JavaScript. This restriction eliminates the threat of cookie theft via cross-site scripting (XSS). However, the cookie remains vulnerable to cross-site tracing (XST) and cross-site request forgery (CSRF) attacks. A cookie is given this characteristic by adding the HttpOnly flag to the cookie.

Same-site cookie

In 2016 Google Chrome version 51 introduced a new kind of cookie with attribute SameSite with possible values of Strict, Lax or None. With attribute SameSite=Strict, the browsers would only send cookies to a target domain that is the same as the origin domain. This would

effectively mitigate cross-site request forgery (CSRF) attacks. With SameSite=Lax, browsers would send cookies with requests to a target domain even it is different from the origin domain, but only for safe requests such as GET (POST is unsafe) and not third-party cookies (inside iframe). Attribute SameSite=None would allow third-party (cross-site) cookies, however, most browsers require secure attribute on SameSite=None cookies.

The Same-site cookie is incorporated into a new RFC draft for "Cookies: HTTP State Management Mechanism" to update RFC 6265 (if approved).

Chrome, Firefox, and Edge started to support Same-site cookies. The key of rollout is the treatment of existing cookies without the SameSite attribute defined, Chrome has been treating those existing cookies as if SameSite=None, this would let all website/applications run as before. Google intended to change that default to SameSite=Lax in Chrome 80 planned to be released in February 2020, but due to potential for breakage of those applications/websites that rely on third-party/cross-site cookies and COVID-19 circumstances, Google postponed this change to Chrome 84.

Supercookie

A supercookie is a cookie with an origin of a top-level domain (such as .com) or a public suffix (such as .co.uk). Ordinary cookies, by contrast, have an origin of a specific domain name, such as example.com.

Supercookies can be a potential security concern and are therefore often blocked by web browsers. If unblocked by the browser, an attacker in control of a malicious website could set a supercookie and potentially disrupt or impersonate legitimate user requests to another website that shares the same top-level domain or public suffix as the malicious website. For example, a supercookie with an origin of .com could maliciously affect a request made to example.com, even if the cookie did not originate from example.com. This can be used to fake logins or change user information.

The Public Suffix List helps to mitigate the risk that supercookies pose. The Public Suffix List is a cross-vendor initiative that aims to provide an accurate and up-to-date list of domain name suffixes. Older versions of browsers may not have an up-to-date list, and will therefore be vulnerable to supercookies from certain domains.

Other uses

The term supercookie is also used for tracking technologies that do not rely on HTTP cookies. Two such supercookie mechanisms were found on Microsoft websites in August 2011: cookie syncing that respawned MUID (machine unique identifier) cookies, and ETag cookies. Due to media attention, Microsoft later disabled this code. In a 2021 blog post, Mozilla used the term supercookie to refer to the use of browser cache as a means of tracking users across sites.

Zombie cookie

A zombie cookie is data and code that has been placed by a web server on a visitor's computer or other device in a hidden location outside the visitor's web browser's dedicated cookie storage location, and that automatically recreates a HTTP cookie as a regular cookie after the original cookie had been deleted. The zombie cookie may be stored in multiple locations, such as Flash Local shared object, HTML5 Web storage, and other client-side and even server-side locations, and when absence is detected in one of the locations, the missing instance is recreated by the JavaScript code using the data stored in other locations.

Structure

A cookie consists of the following components:

Name

Value

Zero or more attributes (name/value pairs). Attributes store information such as the cookie's expiration, domain, and flags (such as Secure and HttpOnly).

Uses

Session management

Cookies were originally introduced to provide a way for users to record items they want to purchase as they navigate throughout a website (a virtual shopping cart or shopping basket). Today, however, the contents of a user's shopping cart are usually stored in a database on the server, rather than in a cookie on the client. To keep track of which user is assigned to which shopping cart, the server sends a cookie to the client that contains a unique session identifier (typically, a long string of random letters and numbers). Because cookies are sent to the server with every request the client makes, that session identifier will be sent back to the server every time the user visits a new page on the website, which lets the server know which shopping cart to display to the user.

Another popular use of cookies is for logging into websites. When the user visits a website's login page, the web server typically sends the client a cookie containing a unique session identifier. When the user successfully logs in, the server remembers that that particular session identifier has been authenticated and grants the user access to its services.

Because session cookies only contain a unique session identifier, this makes the amount of personal information that a website can save about each user virtually limitless—the website is not limited to restrictions concerning how large a cookie can be. Session cookies also help to improve page load times, since the amount of information in a session cookie is small and requires little bandwidth.

Personalization

Cookies can be used to remember information about the user in order to show relevant

content to that user over time. For example, a web server might send a cookie containing the username that was last used to log into a website, so that it may be filled in automatically the next time the user logs in.

Many websites use cookies for personalization based on the user's preferences. Users select their preferences by entering them in a web form and submitting the form to the server. The server encodes the preferences in a cookie and sends the cookie back to the browser. This way, every time the user accesses a page on the website, the server can personalize the page according to the user's preferences. For example, the Google search engine once used cookies to allow users (even non-registered ones) to decide how many search results per page they wanted to see.

Also, DuckDuckGo uses cookies to allow users to set the viewing preferences like colors of the web page.

Tracking

Tracking cookies are used to track users' web browsing habits. This can also be done to some extent by using the IP address of the computer requesting the page or the referer field of the HTTP request header, but cookies allow for greater precision. This can be demonstrated as follows:

If the user requests a page of the site, but the request contains no cookie, the server presumes that this is the first page visited by the user. So the server creates a unique identifier (typically a string of random letters and numbers) and sends it as a cookie back to the browser together with the requested page.

From this point on, the cookie will automatically be sent by the browser to the server every time a new page from the site is requested. The server not only sends the page as usual but also stores the URL of the requested page, the date/time of the request, and the cookie in a

log file.

By analyzing this log file, it is then possible to discover which pages the user has visited, in what sequence, and for how long.

Corporations exploit users' web habits by tracking cookies to collect information about buying habits. The Wall Street Journal found that America's top fifty websites installed an average of sixty-four pieces of tracking technology onto computers, resulting in a total of 3,180 tracking files. The data can then be collected and sold to bidding corporations.

Implementation

Cookies are arbitrary pieces of data, usually chosen and first sent by the web server, and stored on the client computer by the web browser. The browser then sends them back to the server with every request, introducing states (memory of previous events) into otherwise stateless HTTP transactions. Without cookies, each retrieval of a web page or component of a web page would be an isolated event, largely unrelated to all other page views made by the user on the website. Although cookies are usually set by the web server, they can also be set by the client using a scripting language such as JavaScript (unless the cookie's HttpOnly flag is set, in which case the cookie cannot be modified by scripting languages).

The cookie specifications require that browsers meet the following requirements in order to support cookies:

- Can support cookies as large as 4,096 bytes in size.

- Can support at least 50 cookies per domain (i.e. per website).

- Can support at least 3,000 cookies in total.

Setting a cookie

Cookies are set using the Set-Cookie header field, sent in an HTTP response from the web

server. This header field instructs the web browser to store the cookie and send it back in future requests to the server (the browser will ignore this header field if it does not support cookies or has disabled cookies).

As an example, the browser sends its first HTTP request for the homepage of the `www.example.org` website:

The server responds with two Set-Cookie header fields:

The server's HTTP response contains the contents of the website's homepage. But it also instructs the browser to set two cookies. The first, `theme`, is considered to be a session cookie since it does not have an Expires or Max-Age attribute. Session cookies are intended to be deleted by the browser when the browser closes. The second, `sessionToken`, is considered to be a persistent cookie since it contains an Expires attribute, which instructs the browser to delete the cookie at a specific date and time.

Next, the browser sends another request to visit the `spec.html` page on the website. This request contains a Cookie header field, which contains the two cookies that the server instructed the browser to set:

This way, the server knows that this HTTP request is related to the previous one. The server would answer by sending the requested page, possibly including more Set-Cookie header fields in the HTTP response in order to instruct the browser to add new cookies, modify existing cookies, or remove existing cookies. To remove a cookie, the server must include a Set-Cookie header field with an expiration date in the past.

The value of a cookie may consist of any printable ASCII character (! through ~, Unicode `\u0021` through `\u007E`) excluding `,` and `;` and whitespace characters. The name of a cookie excludes the same characters, as well as `=`, since that is the delimiter between the name and

value. The cookie standard RFC 2965 is more restrictive but not implemented by browsers.

The term cookie crumb is sometimes used to refer to a cookie's name–value pair.

Cookies can also be set by scripting languages such as JavaScript that run within the browser.

In JavaScript, the object `document.cookie` is used for this purpose. For example, the instruction `document.cookie = "temperature=20"` creates a cookie of name `temperature` and value `20`.

Cookie attributes

In addition to a name and value, cookies can also have one or more attributes. Browsers do not include cookie attributes in requests to the server—they only send the cookie's name and value. Cookie attributes are used by browsers to determine when to delete a cookie, block a cookie or whether to send a cookie to the server.

Domain and Path

The Domain and Path attributes define the scope of the cookie. They essentially tell the browser what website the cookie belongs to. For security reasons, cookies can only be set on the current resource's top domain and its subdomains, and not for another domain and its subdomains. For example, the website `example.org` cannot set a cookie that has a domain of `foo.com` because this would allow the website `example.org` to control the cookies of the domain `foo.com`.

If a cookie's Domain and Path attributes are not specified by the server, they default to the domain and path of the resource that was requested. However, in most browsers there is a difference between a cookie set from `foo.com` without a domain, and a cookie set with the `foo.com` domain. In the former case, the cookie will only be sent for requests to `foo.com`, also known as a host-only cookie. In the latter case, all subdomains are also included (for example, `docs.foo.com`). A notable exception to this general rule is Edge prior to Windows 10 RS3 and

Internet Explorer prior to IE 11 and Windows 10 RS4 (April 2018), which always sends cookies to subdomains regardless of whether the cookie was set with or without a domain.

Below is an example of some Set-Cookie header fields in the HTTP response of a website after a user logged in. The HTTP request was sent to a webpage within the docs.foo.com subdomain:

The first cookie, LSID, has no Domain attribute, and has a Path attribute set to /accounts. This tells the browser to use the cookie only when requesting pages contained in docs.foo.com/accounts (the domain is derived from the request domain). The other two cookies, HSID and SSID, would be used when the browser requests any subdomain in .foo.com on any path (for example www.foo.com/bar). The prepending dot is optional in recent standards, but can be added for compatibility with RFC 2109 based implementations.

Expires and Max-Age

The Expires attribute defines a specific date and time for when the browser should delete the cookie. The date and time are specified in the form Wdy, DD Mon YYYY HH:MM:SS GMT, or in the form Wdy, DD Mon YY HH:MM:SS GMT for values of YY where YY is greater than or equal to 0 and less than or equal to 69.

Alternatively, the Max-Age attribute can be used to set the cookie's expiration as an interval of seconds in the future, relative to the time the browser received the cookie. Below is an example of three Set-Cookie header fields that were received from a website after a user logged in:

The first cookie, lu, is set to expire sometime on 15 January 2013. It will be used by the client browser until that time. The second cookie, made_write_conn, does not have an expiration date, making it a session cookie. It will be deleted after the user closes their browser. The third

cookie, `reg_fb_gate`, has its value changed to `deleted`, with an expiration time in the past. The browser will delete this cookie right away because its expiration time is in the past. Note that cookie will only be deleted if the domain and path attributes in the `Set-Cookie` field match the values used when the cookie was created.

As of 2016 Internet Explorer did not support `Max-Age`.

Secure and `HttpOnly`

The `Secure` and `HttpOnly` attributes do not have associated values. Rather, the presence of just their attribute names indicates that their behaviors should be enabled.

The `Secure` attribute is meant to keep cookie communication limited to encrypted transmission, directing browsers to use cookies only via secure/encrypted connections. However, if a web server sets a cookie with a `secure` attribute from a non-secure connection, the cookie can still be intercepted when it is sent to the user by man-in-the-middle attacks. Therefore, for maximum security, cookies with the `Secure` attribute should only be set over a secure connection.

The `HttpOnly` attribute directs browsers not to expose cookies through channels other than HTTP (and HTTPS) requests. This means that the cookie cannot be accessed via client-side scripting languages (notably JavaScript), and therefore cannot be stolen easily via cross-site scripting (a pervasive attack technique).

Browser settings

Most modern browsers support cookies and allow the user to disable them. The following are common options:

To enable or disable cookies completely, so that they are always accepted or always blocked.

To view and selectively delete cookies using a cookie manager.

To fully wipe all private data, including cookies.

Add-on tools for managing cookie permissions also exist.

Third-party cookie

Cookies have some important implications for the privacy and anonymity of web users. While cookies are sent only to the server setting them or a server in the same Internet domain, a web page may contain images or other components stored on servers in other domains. Cookies that are set during retrieval of these components are called third-party cookies. A third-party cookie, belongs to a domain different from the one shown in the address bar. This sort of cookie typically appears when web pages feature content from external websites, such as banner advertisements. This opens up the potential for tracking the user's browsing history and is used by advertisers to serve relevant advertisements to each user.

As an example, suppose a user visits www.example.org. This website contains an advertisement from ad.foxytracking.com, which, when downloaded, sets a cookie belonging to the advertisement's domain (ad.foxytracking.com). Then, the user visits another website, www.foo.com, which also contains an advertisement from ad.foxytracking.com and sets a cookie belonging to that domain (ad.foxytracking.com). Eventually, both of these cookies will be sent to the advertiser when loading their advertisements or visiting their website. The advertiser can then use these cookies to build up a browsing history of the user across all the websites that have ads from this advertiser, through the use of the HTTP referer header field. As of 2014, some websites were setting cookies readable for over 100 third-party domains. On average, a single website was setting 10 cookies, with a maximum number of cookies (first- and third-party) reaching over 800.

The older standards for cookies, RFC 2109 and RFC 2965, recommend that browsers should protect user privacy and not allow sharing of cookies between servers by default. However,

the newer standard, RFC 6265, explicitly allows user agents to implement whichever third-party cookie policy they wish. Most modern web browsers contain privacy settings that can block third-party cookies. Since 2020, Apple Safari, Firefox, and Brave block all third-party cookies by default. Safari allows embedded sites to use Storage Access API to request permission to set first-party cookies. In May 2020, Google Chrome 83 introduced new features to block third-party cookies by default in its Incognito mode for private browsing, making blocking optional during normal browsing. The same update also added an option to block first-party cookies. In April 2024, Chrome postponed third-party cookie blocking by default to 2025. In July 2024, Google announced plan to avoid blocking third-party cookies by default and instead prompt users to allow third-party cookies.

Privacy

The possibility of building a profile of users is a privacy threat, especially when tracking is done across multiple domains using third-party cookies. For this reason, some countries have legislation about cookies.

Website operators who do not disclose third-party cookie use to consumers run the risk of harming consumer trust if cookie use is discovered. Having clear disclosure (such as in a privacy policy) tends to eliminate any negative effects of such cookie discovery.

The government of the United States set strict rules on setting cookies in 2000 after it was disclosed that the White House drug policy office used cookies to track computer users viewing its online anti-drug advertising. In 2002, privacy activist Daniel Brandt found that the CIA had been leaving persistent cookies on computers that had visited its website. When notified it was violating policy, CIA stated that these cookies were not intentionally set and stopped setting them. On 25 December 2005, Brandt discovered that the National Security Agency (NSA) had been leaving two persistent cookies on visitors' computers due to a software upgrade. After being informed, the NSA immediately disabled the cookies.

EU cookie directive

In 2002, the European Union launched the Directive on Privacy and Electronic Communications (e-Privacy Directive), a policy requiring end users' consent for the placement of cookies, and similar technologies for storing and accessing information on users' equipment. In particular, Article 5 Paragraph 3 mandates that storing technically unnecessary data on a user's computer can only be done if the user is provided information about how this data is used, and the user is given the possibility of denying this storage operation. The Directive does not require users to authorise or be provided notice of cookie usage that are functionally required for delivering a service they have requested, for example to retain settings, store log-in sessions, or remember what is in a user's shopping basket.

In 2009, the law was amended by Directive 2009/136/EC, which included a change to Article 5, Paragraph 3. Instead of having an option for users to opt out of cookie storage, the revised Directive requires consent to be obtained for cookie storage. The definition of consent is cross-referenced to the definition in European data protection law, firstly the Data Protection Directive 1995 and subsequently the General Data Protection Regulation (GDPR). As the definition of consent was strengthened in the text of the GDPR, this increased the quality of consent required by those storing and accessing information such as cookies on users' devices. In a case decided under the Data Protection Directive however, the Court of Justice of the European Union later confirmed however that the previous law implied the same strong quality of consent as the current instrument. In addition to the requirement of consent which stems from storing or accessing information on a user's terminal device, the information in many cookies will be considered personal data under the GDPR alone, and will require a legal basis to process. This has been the case since the 1995 Data Protection Directive, which used an identical definition of personal data, although the GDPR in interpretative Recital 30 clarifies that cookie identifiers are included. While not all data processing under the GDPR requires

consent, the characteristics of behavioural advertising mean that it is difficult or impossible to justify under any other ground.

Consent under the combination of the GDPR and e-Privacy Directive has to meet a number of conditions in relation to cookies. It must be freely given and unambiguous: pre-ticked boxes were banned under both the Data Protection Directive 1995 and the GDPR (Recital 32). The GDPR is specific that consent must be as 'easy to withdraw as to give', meaning that a reject-all button must be as easy to access in terms of clicks and visibility as an 'accept all' button. It must be specific and informed, meaning that consent relates to particular purposes for the use of this data, and all organisations seeking to use this consent must be specifically named. The Court of Justice of the European Union has also ruled that consent must be 'efficient and timely', meaning that it must be gained before cookies are laid and data processing begins instead of afterwards.

The industry's response has been largely negative. Robert Bond of the law firm Speechly Bircham describes the effects as "far-reaching and incredibly onerous" for "all UK companies". Simon Davis of Privacy International argues that proper enforcement would "destroy the entire industry". However, scholars note that the onerous nature of cookie pop-ups stems from an attempt to continue to operate a business model through convoluted requests that may be incompatible with the GDPR.

Academic studies and regulators both describe widespread non-compliance with the law. A study scraping 10,000 UK websites found that only 11.8% of sites adhered to minimal legal requirements, with only 33.4% of websites studied providing a mechanism to reject cookies that was as easy to use as accepting them. A study of 17,000 websites found that 84% of sites breached this criterion, finding additionally that many laid third party cookies with no notice at all. The UK regulator, the Information Commissioner's Office, stated in 2019 that the industry's 'Transparency and Consent Framework' from the advertising technology group the Interactive Advertising Bureau was 'insufficient to ensure transparency and fair processing of the

personal data in question and therefore also insufficient to provide for free and informed consent, with attendant implications for PECR [e-Privacy] compliance.' Many companies that sell compliance solutions (Consent Management Platforms) permit them to be configured in manifestly illegal ways, which scholars have noted creates questions around the appropriate allocation of liability.

A W3C specification called P3P was proposed for servers to communicate their privacy policy to browsers, allowing automatic, user-configurable handling. However, few websites implement the specification, and the W3C has discontinued work on the specification.

Third-party cookies can be blocked by most browsers to increase privacy and reduce tracking by advertising and tracking companies without negatively affecting the user's web experience on all sites. Some sites operate 'cookie walls', which make access to a site conditional on allowing cookies either technically in a browser, through pressing 'accept', or both. In 2020, the European Data Protection Board, composed of all EU data protection regulators, stated that cookie walls were illegal. In order for consent to be freely given, access to services and functionalities must not be made conditional on the consent of a user to the storing of information, or gaining of access to information already stored, in the terminal equipment of a user (so called cookie walls). Many advertising operators have an opt-out option to behavioural advertising, with a generic cookie in the browser stopping behavioural advertising. However, this is often ineffective against many forms of tracking, such as first-party tracking that is growing in popularity to avoid the impact of browsers blocking third party cookies. Furthermore, if such a setting is more difficult to place than the acceptance of tracking, it remains in breach of the conditions of the e-Privacy Directive.

Cookie theft and session hijacking

Most websites use cookies as the only identifiers for user sessions, because other methods of

identifying web users have limitations and vulnerabilities. If a website uses cookies as session identifiers, attackers can impersonate users' requests by stealing a full set of victims' cookies. From the web server's point of view, a request from an attacker then has the same authentication as the victim's requests; thus the request is performed on behalf of the victim's session.

Listed here are various scenarios of cookie theft and user session hijacking (even without stealing user cookies) that work with websites relying solely on HTTP cookies for user identification.

Network eavesdropping

Traffic on a network can be intercepted and read by computers on the network other than the sender and receiver (particularly over unencrypted open Wi-Fi). This traffic includes cookies sent on ordinary unencrypted HTTP sessions. Where network traffic is not encrypted, attackers can therefore read the communications of other users on the network, including HTTP cookies as well as the entire contents of the conversations, for the purpose of a man-in-the-middle attack.

An attacker could use intercepted cookies to impersonate a user and perform a malicious task, such as transferring money out of the victim's bank account.

This issue can be resolved by securing the communication between the user's computer and the server by employing Transport Layer Security (HTTPS protocol) to encrypt the connection. A server can specify the Secure flag while setting a cookie, which will cause the browser to send the cookie only over an encrypted channel, such as a TLS connection.

Publishing false sub-domain: DNS cache poisoning

If an attacker is able to cause a DNS server to cache a fabricated DNS entry (called DNS cache poisoning), then this could allow the attacker to gain access to a user's cookies. For example,

an attacker could use DNS cache poisoning to create a fabricated DNS entry of f12345.www.example.com that points to the IP address of the attacker's server. The attacker can then post an image URL from his own server (for example, http://f12345.www.example.com/img_4_cookie.jpg). Victims reading the attacker's message would download this image from f12345.www.example.com. Since f12345.www.example.com is a sub-domain of www.example.com, victims' browsers would submit all example.com-related cookies to the attacker's server.

If an attacker is able to accomplish this, it is usually the fault of the Internet service providers for not properly securing their DNS servers. However, the severity of this attack can be lessened if the target website uses secure cookies. In this case, the attacker would have the extra challenge of obtaining the target website's TLS certificate from a certificate authority, since secure cookies can only be transmitted over an encrypted connection. Without a matching TLS certificate, victims' browsers would display a warning message about the attacker's invalid certificate, which would help deter users from visiting the attacker's fraudulent website and sending the attacker their cookies.

Cross-site scripting: cookie theft

Cookies can also be stolen using a technique called cross-site scripting. This occurs when an attacker takes advantage of a website that allows its users to post unfiltered HTML and JavaScript content. By posting malicious HTML and JavaScript code, the attacker can cause the victim's web browser to send the victim's cookies to a website the attacker controls.

As an example, an attacker may post a message on www.example.com with the following link:

When another user clicks on this link, the browser executes the piece of code within the onclick attribute, thus replacing the string document.cookie with the list of cookies that are accessible from the current page. As a result, this list of cookies is sent to the attacker.com

server. If the attacker's malicious posting is on an HTTPS website <https://www.example.com>, secure cookies will also be sent to attacker.com in plain text.

It is the responsibility of the website developers to filter out such malicious code.

Such attacks can be mitigated by using HttpOnly cookies. These cookies will not be accessible by client-side scripting languages like JavaScript, and therefore, the attacker will not be able to gather these cookies.

Cross-site scripting: proxy request

In older versions of many browsers, there were security holes in the implementation of the XMLHttpRequest API. This API allows pages to specify a proxy server that would get the reply, and this proxy server is not subject to the same-origin policy. For example, a victim is reading an attacker's posting on www.example.com, and the attacker's script is executed in the victim's browser. The script generates a request to www.example.com with the proxy server attacker.com. Since the request is for www.example.com, all example.com cookies will be sent along with the request, but routed through the attacker's proxy server. Hence, the attacker would be able to harvest the victim's cookies.

This attack would not work with secure cookies, since they can only be transmitted over HTTPS connections, and the HTTPS protocol dictates end-to-end encryption (i.e. the information is encrypted on the user's browser and decrypted on the destination server). In this case, the proxy server would only see the raw, encrypted bytes of the HTTP request.

Cross-site request forgery

For example, Bob might be browsing a chat forum where another user, Mallory, has posted a message. Suppose that Mallory has crafted an HTML image element that references an action on Bob's bank's website (rather than an image file), e.g.,

If Bob's bank keeps his authentication information in a cookie, and if the cookie hasn't expired, then the attempt by Bob's browser to load the image will submit the withdrawal form with his cookie, thus authorizing a transaction without Bob's approval.

Cookiejacking

Cookiejacking is an attack against Internet Explorer which allows the attacker to steal session cookies of a user by tricking a user into dragging an object across the screen. Microsoft deemed the flaw low-risk because of "the level of required user interaction", and the necessity of having a user already logged into the website whose cookie is stolen. Despite this, a researcher tried the attack on 150 of their Facebook friends and obtained cookies of 80 of them via social engineering.

Drawbacks of cookies

Besides privacy concerns, cookies also have some technical drawbacks. In particular, they do not always accurately identify users, they can be used for security attacks, and they are often at odds with the Representational State Transfer (REST) software architectural style.

Inaccurate identification

If more than one browser is used on a computer, each usually has a separate storage area for cookies. Hence, cookies do not identify a person, but a combination of a user account, a computer, and a web browser. Thus, anyone who uses multiple accounts, computers, or browsers has multiple sets of cookies.

Likewise, cookies do not differentiate between multiple users who share the same user account, computer, and browser.

Alternatives to cookies

Some of the operations that can be done using cookies can also be done using other mechanisms.

Authentication and session management

JSON Web Tokens

A JSON Web Token (JWT) is a self-contained packet of information that can be used to store user identity and authenticity information. This allows them to be used in place of session cookies. Unlike cookies, which are automatically attached to each HTTP request by the browser, JWTs must be explicitly attached to each HTTP request by the web application.

HTTP authentication

The HTTP protocol includes the basic access authentication and the digest access authentication protocols, which allow access to a web page only when the user has provided the correct username and password. If the server requires such credentials for granting access to a web page, the browser requests them from the user and, once obtained, the browser stores and sends them in every subsequent page request. This information can be used to track the user.

URL (query string)

The query string part of the URL is the part that is typically used for this purpose, but other parts can be used as well. The Java Servlet and PHP session mechanisms both use this method if cookies are not enabled.

This method consists of the web server appending query strings containing a unique session identifier to all the links inside of a web page. When the user follows a link, the browser sends the query string to the server, allowing the server to identify the user and maintain state.

These kinds of query strings are very similar to cookies in that both contain arbitrary pieces of information chosen by the server and both are sent back to the server on every request. However, there are some differences. Since a query string is part of a URL, if that URL is later reused, the same attached piece of information will be sent to the server, which could lead to confusion. For example, if the preferences of a user are encoded in the query string of a URL and the user sends this URL to another user by e-mail, those preferences will be used for that other user as well.

Moreover, if the same user accesses the same page multiple times from different sources, there is no guarantee that the same query string will be used each time. For example, if a user visits a page by coming from a page internal to the site the first time, and then visits the same page by coming from an external search engine the second time, the query strings would likely be different. If cookies were used in this situation, the cookies would be the same.

Other drawbacks of query strings are related to security. Storing data that identifies a session in a query string enables session fixation attacks, referer logging attacks and other security exploits. Transferring session identifiers as HTTP cookies is more secure.

Hidden form fields

Another form of session tracking is to use web forms with hidden fields. This technique is very similar to using URL query strings to hold the information and has many of the same advantages and drawbacks. In fact, if the form is handled with the HTTP GET method, then this technique is similar to using URL query strings, since the GET method adds the form fields to the URL as a query string. But most forms are handled with HTTP POST, which causes the form information, including the hidden fields, to be sent in the HTTP request body, which is neither part of the URL, nor of a cookie.

This approach presents two advantages from the point of view of the tracker. First, having the tracking information placed in the HTTP request body rather than in the URL means it will not

be noticed by the average user. Second, the session information is not copied when the user copies the URL (to bookmark the page or send it via email, for example).

window.name DOM property

All current web browsers can store a fairly large amount of data (2–32 MB) via JavaScript using the DOM property `window.name`. This data can be used instead of session cookies. The technique can be coupled with JSON/JavaScript objects to store complex sets of session variables on the client side.

The downside is that every separate window or tab will initially have an empty `window.name` property when opened.

In some respects, this can be more secure than cookies because its contents are not automatically sent to the server on every request like cookies are, so it is not vulnerable to network cookie sniffing attacks.

Tracking

IP address

Some users may be tracked based on the IP address of the computer requesting the page. The server knows the IP address of the computer running the browser (or the proxy, if any is used) and could theoretically link a user's session to this IP address.

However, IP addresses are generally not a reliable way to track a session or identify a user. Many computers designed to be used by a single user, such as office PCs or home PCs, are behind a network address translator (NAT). This means that several PCs will share a public IP address. Furthermore, some systems, such as Tor, are designed to retain Internet anonymity, rendering tracking by IP address impractical, impossible, or a security risk.

ETag

Because ETags are cached by the browser, and returned with subsequent requests for the same resource, a tracking server can simply repeat any ETag received from the browser to ensure an assigned ETag persists indefinitely (in a similar way to persistent cookies). Additional caching header fields can also enhance the preservation of ETag data.

ETags can be flushed in some browsers by clearing the browser cache.

Browser cache

The browser cache can also be used to store information that can be used to track individual users. This technique takes advantage of the fact that the web browser will use resources stored within the cache instead of downloading them from the website when it determines that the cache already has the most up-to-date version of the resource.

For example, a website could serve a JavaScript file with code that sets a unique identifier for the user (for example, `var userId = 3243242;`). After the user's initial visit, every time the user accesses the page, this file will be loaded from the cache instead of downloaded from the server. Thus, its content will never change.

Browser fingerprint

A browser fingerprint is information collected about a browser's configuration, such as version number, screen resolution, and operating system, for the purpose of identification. Fingerprints can be used to fully or partially identify individual users or devices even when cookies are turned off.

Basic web browser configuration information has long been collected by web analytics services in an effort to accurately measure real human web traffic and discount various forms of click fraud. With the assistance of client-side scripting languages, collection of much more esoteric parameters is possible. Assimilation of such information into a single string constitutes a device fingerprint. In 2010, EFF measured at least 18.1 bits of entropy possible

from browser fingerprinting. Canvas fingerprinting, a more recent technique, claims to add another 5.7 bits.

Web storage

Some web browsers support persistence mechanisms which allow the page to store the information locally for later use.

The HTML5 standard (which most modern web browsers support to some extent) includes a JavaScript API called Web storage that allows two types of storage: local storage and session storage. Local storage behaves similarly to persistent cookies while session storage behaves similarly to session cookies, except that session storage is tied to an individual tab/window's lifetime (AKA a page session), not to a whole browser session like session cookies.

Internet Explorer supports persistent information in the browser's history, in the browser's favorites, in an XML store ("user data"), or directly within a web page saved to disk.

Some web browser plugins include persistence mechanisms as well. For example, Adobe Flash has Local shared object and Microsoft Silverlight has Isolated storage.

See also

HTTP Strict Transport Security § Privacy issues

Secure cookie – Type of HTTP cookie

Session (computer science) – Temporary context for interactive information interchange
Pages displaying short descriptions of redirect targets

References

Sources

Anonymous, 2011. Cookiejacking Attack Steals Website Access Credentials. Informationweek - Online, pp. Informationweek - Online, 26 May 2011.

External links

[RFC 6265, the current official specification for HTTP cookies](#)

[HTTP cookies, Mozilla Developer Network](#)

[Using cookies via ECMAScript, Mozilla Developer Network](#)

[How Internet Cookies Work at HowStuffWorks](#)

[Cookies at the Electronic Privacy Information Center \(EPIC\)](#)

[Mozilla Knowledge-Base: Cookies](#)

[Cookie Domain, explain in detail how cookie domains are handled in current major browsers](#)

[Cookie Stealing - Michael Pound](#)

[Check cookies for compliance with EU cookie directive](#)